Practical No. 1

Aim:- Write a program to check the current CPU temperature and condition using raspberry pi.

Code:-

```
import psutil
import time

print("====================================")
print("     Raspberry Pi System Monitor")
print("====================================")

try:
    while True:
        # Read CPU usage
        cpu_usage = psutil.cpu_percent(interval=1)

        # Read memory usage
        memory = psutil.virtual_memory()
        memory_usage = memory.percent

        # Read CPU temperature
        temperatures = psutil.sensors_temperatures()
        cpu_temp = None

        if "cpu_thermal" in temperatures and temperatures["cpu_thermal"]:
            cpu_temp = temperatures["cpu_thermal"][0].current
        elif "thermal_zone0" in temperatures and temperatures["thermal_zone0"]:
            cpu_temp = temperatures["thermal_zone0"][0].current
```

```
# Display system information
print("-----------------------------------")
print("CPU Usage       :", cpu_usage, "%")
print("Memory Usage    :", memory_usage, "%")

if cpu_temp is not None:
    print("CPU Temperature :", cpu_temp, "°C")
else:
    print("CPU Temperature : Not Available")

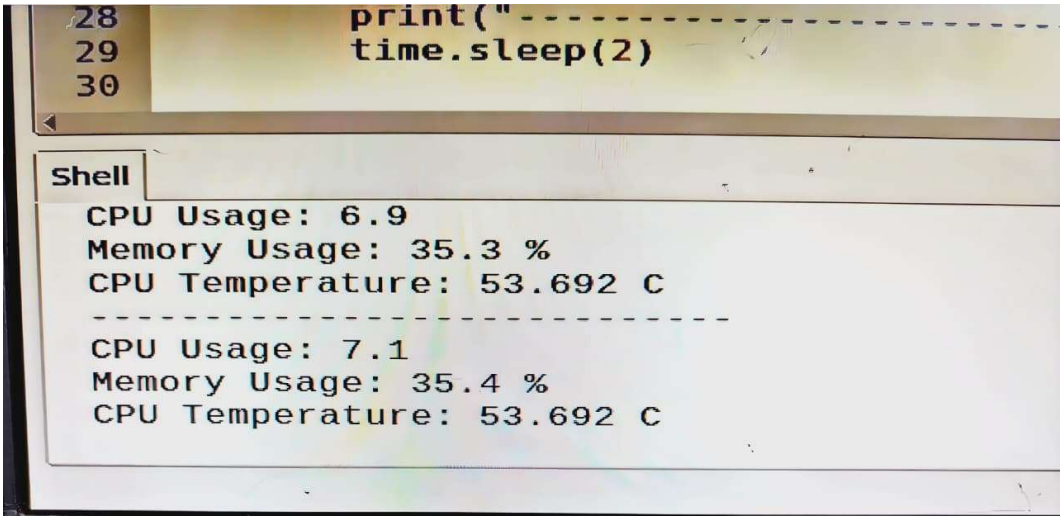
print("-----------------------------------")

time.sleep(2)
```

Open cmd and type:

- sudo apt update
- sudo apt install python3 python3-pip -y
- python3 -c "import psutil; print('CPU Usage:', psutil.cpu_percent(interval=1), '%')"
- python3 -c "import psutil; print('CPU Usage:', psutil.cpu_percent(interval=1), '%')"

Expected Output:-



Practical No.2

Aim:- Install Windows IoT 10 core on Raspberry pi and run the demo program.

Requirements

- Raspberry Pi 3B+
- microSD card, preferably **8 GB or larger**
- Windows PC
- SD-card reader
- Raspberry Pi power supply
- HDMI monitor and keyboard/network connection

Step 1: Download Windows 10 IoT Core

On the Windows PC, obtain the appropriate **Windows 10 IoT Core image/package for Raspberry Pi 3** from Microsoft's Windows IoT documentation/archive.

The image is normally provided as an .ffu file or through an installer that prepares the SD card.

Step 2: Prepare the microSD card

Insert the microSD card into the Windows PC.

Use Microsoft's supported IoT Core installation tools to write the image to the card.

If using a command-line FFU workflow, the basic operation is to apply the .ffu image to the SD card using Microsoft's deployment tools.

Warning: Writing the image will erase the contents of the microSD card.

Step 3: Insert the card into Raspberry Pi 3B+

1. Turn off the Raspberry Pi.
2. Insert the prepared microSD card.
3. Connect HDMI, keyboard and network.
4. Connect the power supply.
5. Wait for Windows 10 IoT Core to boot.

The first boot can take several minutes.

Step 4: Find the Raspberry Pi's IP address

Connect the Pi to your network using Ethernet or Wi-Fi if supported by your image.

From the Windows PC, you can check the network or use:

ping raspberrypi

If that does not work, find the Pi's IP address from your router.

Step 5: Open Windows Device Portal

On the Windows PC, open:
http://<Raspberry-Pi-IP>:8080

For example:
http://192.168.1.25:8080

Windows IoT Core's **Device Portal** allows you to manage the Raspberry Pi and deploy applications.

Step 6: Run the demo application

Windows IoT Core provides sample applications such as:

- **Hello Blinky**
- GPIO/LED samples
- Basic Universal Windows Platform (UWP) applications

For a Raspberry Pi GPIO demonstration, **Hello Blinky** is a suitable starting example. The basic flow is:

Windows PC
  |
  | Deploy UWP application
  ↓
Raspberry Pi 3B+
  |
  ↓
Windows 10 IoT Core
  |
  ↓
GPIO
  |
  ↓
LED ON/OFF

Step 7: Example demo — LED blinking

Connect an LED with a suitable resistor to a GPIO pin, for example:

Raspberry Pi 3B+

GPIO 17 (Pin 11) ── 220Ω ── LED ── GND (Pin 6)

A UWP application can then control GPIO 17:

using Windows.Devices.Gpio;

```
using System.Threading.Tasks;

GpioPin ledPin;

private void InitGPIO()
{
   var gpio = GpioController.GetDefault();

   if (gpio == null)
      return;
   ledPin = gpio.OpenPin(17);
   ledPin.SetDriveMode(GpioPinDriveMode.Output);
   ledPin.Write(GpioPinValue.Low);
}

private async Task BlinkLED()
{
   while (true)
   {
      ledPin.Write(GpioPinValue.High);
      await Task.Delay(1000);

      ledPin.Write(GpioPinValue.Low);
      await Task.Delay(1000);
   }
}
```

This produces:

```
LED ON
 ↓ 1 second
LED OFF
 ↓ 1 second
LED ON
 ↓
...
```

Expected result

After deploying and running the demo:

```
Windows 10 IoT Core
      ↓
 Raspberry Pi 3B+
      ↓
   GPIO 17
      ↓
     LED
      ↓
ON ── OFF ── ON ── OFF
```